Sounding Rocket Air Brakes Control Algorithm Analysis

By Igby McLaverty

Github repo: https://github.com/IgbyMcLaverty/MMA3001Project/tree/main

Problem Context

Monash High Powered Rocketry (MHPR) is one of the engineering student teams at Monash University. They create low altitude sounding rockets which enter into competitions which are split into two components, the design and engineering of the rocket, and the performance of a single flight made at the competition. The aim of the flight is to reach a target apogee of 10,000 ft which is 3,048 meters. This target value has been used throughout this project. Points are award based on how far above or below this altitude the rocket reaches. To fine tune the apogee of their rockets, MHPR add deployable Air Brakes to their rockets.

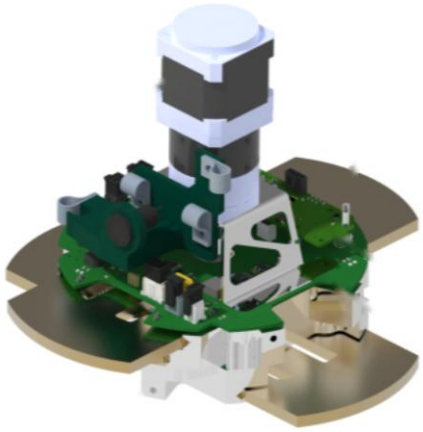


Figure 1: Physical Air Brakes assembly

Project Meridian was MHPRs most recent competition rocket fitted with Air Brakes and has been used as a case study for this project. The Air Brakes system, pictured in figure 1, consists of four flaps which extend out the air frame to increase the drag on the rocket during flight. The extension of these flaps can be controlled with high precision, however the control algorithm which calculates what extension is optimal to lower the apogee of the rocket to the target requires evaluation before a launch. The exact trajectory cannot be predicted before launch because the impulse delivered by the rocket motor varies between launches, meaning whatever control algorithm gets put in place needs to be adaptable to the current conditions of the rocket.

The control algorithm needs to run on an STM32 H7 microprocessor which creates processing power and storage space limitations on the control algorithm. Additionally the information available is limited and noisy, only having access to data from the onboard barometers and accelerometers and information which can be measured while the rocket is still grounded, such as ground temperature, pressure and lapse rate.

The aim of this project was to create a framework to quickly evaluate the effectiveness of different control algorithms without needing to run hours' worth of simulations.

MHPR has developed a highly accurate six degree of freedom (6DOF) simulator called SATURN. It takes in the atmospheric conditions and a model of the rocket and predicts it's trajectory. Data from SATURN has been used as the ground truth for this project.

1DOF Simulation

SATURN can model Air Brakes control algorithms; however, it takes approximately 30 seconds to perform a single simulation meaning it's not well suited for quickly evaluating them. Since the training for some algorithms uses the simulation directly and would require hundreds of runs, there was a need for a faster simulation to run these algorithms on.

A one degree of freedom (1DOF) simulation was chosen to be developed as this is the simplest model which would produce reasonable results.

Assumptions and Simplifications

Table 1: 1DOF simulations assumptions

Assumption Number	Assumption
#1	Drag force follows the drag equation
#2	Air is an ideal gas
#3	Pressure changes according to the barometric formula
#4	Acceleration due to gravity is constant at 9.81 m/s^2
#5	Lapse rate is constant
#6	Rocket pitch relative to the air remains zero throughout flight
#7	Drag coefficient is independent of Mach number
#8	Drag coefficient changes linearly with extension
#9	Simulation only begins after burnout
#10	Air Brakes extend instantly

In creating the simulation, many assumptions and simplifications, detailed in table 1, were made to keep the model simple and fast. CFD data exists for Project Meridian which could remove assumptions 7 and 8, however it was found implementing 2D linear interpolation for coefficient of drag significantly increased the execution time without providing an improvement to accuracy.

Methods

Table 2: Initial conditions and parameters of the simulation

Input	Units	Description
Initial Altitude	Meters	Altitude at which the rocket motor no longer produces thrust (burnout)
Initial Velocity	Meters per second	Velocity at burnout
Sea Level Pressure	Pascal	The pressure at sea level
Sea Level Temperature	Kelvin	The temperature at sea level
Lapse Rate	Kelvin per meter	Amount the temperature increases or decreases with altitude
Rocket Mass	Kilograms	The mass of the rocket at burnout (dry mass)
0% Drag Coefficient	Dimensionless	The coefficient of drag when the flaps are retracted
100% Drag Coefficient	Dimensionless	The coefficient of drag when the flaps are extended
0% Area	Meters squared	The cross-sectional area when the flaps are retracted
100% Area	Meters squared	The cross-sectional area when the flaps are extended
Controller	Function	The control algorithm to be used in the simulation

The available inputs to the simulation are shown in table 2. Mass, drag coefficients and areas were all kept constant between trails. The values used correspond to Project Meridian.

$$D = \frac{1}{2} \cdot \rho \cdot c_d \cdot A \cdot V^2 \qquad P = P_0 \cdot \left(1 + \frac{L \cdot h}{T_0}\right)^{-\frac{g \cdot M}{R_0 \cdot L}} \qquad \rho = \frac{P}{R_{air} \cdot T}$$

Eq 1: Drag Equation | Eq 2: Barometric Formula | Eq 3: Ideal gas law rearranged for density

The simulation used a fixed time step approach to calculate the trajectory of the rocket. To calculate the air density required by the drag equation (Eq 1), the temperature was found using the current altitude assuming a constant lapse rate from sea level and pressure was calculated using the barometric formula (Eq 2), this was fed into the ideal gas law (Eq 3) to solve for air density. Based on the current extension value, the drag coefficient and cross-sectional area were linearly interpolated between the provided maximums and minimums. The drag and weight forces were then used to find the net acceleration.

An RK2 implementation between the acceleration and velocity was used to allow for a larger time step and thus decrease execution time. The calculated acceleration is used to find velocity and altitude at an intermediate point where the process is repeated, and a new acceleration is calculated. This new acceleration is then applied across the entire timestep. Integrating from velocity to altitude used Eulers method rather than RK2 for simplicity. The current rocket state was then fed into the control algorithm to get the extension for the next timestep of the simulation.

The simulation continued until the velocity became negative, meaning the rocket had reached apogee. The altitude, velocity, acceleration, temperature, pressure and air density at each timestep were saved and returned at the end of the simulation.

Optimisation

To find an appropriate value for the timestep which ensured the simulation converged sufficiently, a function was produced which automatically iterated through different timesteps. The function began with a supplied timestep, it ran the simulation and recorded the apogee reached. The timestep was then halved and the simulation was run again with the same initial conditions. The time step continued to be halved until the change in apogee between two consecutive runs was less than a given tolerance.

A tolerance of 1 meter was chosen since the root mean squared error (RMSE) of the simulation was on the order of 50 meters even having converged to within 0.1 meters. Hence converging more than 1 meter would have been wasting processing time as the truncation error was already dwarfed by other sources of error.

Execution time was measured using the timeit library and cProfile. All timing was performed on the same hardware under similar conditions and for timeit, the simulation was run 100 times. Early versions of the simulation used Eulers method but required a small timestep of approximately 0.005 seconds and an average execution time of 0.0407 seconds to converge within 1 meter. Thus, to improve this the RK2 implementation was developed which required a timestep of approximately 0.01 seconds and had an average execution time of 0.0365 seconds. For the same convergence RK2 was able to execute faster and was thus chosen as the method for the final implementation.

RK4 was considered but not implemented because it would only improve the average execution time if the timestep was increased. A timestep larger than 0.01 seconds was not desirable for this application since it would limit the frequency at which control algorithm could be simulated. In the real-life system, the control algorithm runs at approximately 100Hz. The performance of the control algorithm may depend on having that high frequency and thus for this simulation to account for algorithms which require a high execution rate, the timestep was limited to a maximum of 0.01 seconds. If the timestep was kept the same, an RK4 implementation would just increase the convergence at the cost of execution time. Since the truncation error was already small compared with other error sources, this would provide little benefit.

Validation and Accuracy

Data to validate the simulation was generated by SATURN. 400 simulations were run in total, 200 with the Air Brakes extension fixed at 0% and 200 with a randomly chosen extension which were maintained for the entire flight. Across all simulations, there was variation in the atmospheric conditions and motor impulse. This variation meant residuals from the apogee error could be calculated and plotted ensure each input was accounted for correctly in the 1DOF simulation.

The data from Saturn was provided as CSV files which contained the rocket state at every timestep in the simulation. SATURN simulations run for the entire flight of the rocket, including the burn and decent phases. Since modelling these was outside the scope of the 1DOF simulation, timesteps during these phases were removed along with extraneous variable which were not needed to compare against the 1DOF.

The data was split into training and test data. The first 150 simulation files from both the with Air Brakes and without Air Brakes datasets were used as training data. While the remaining 50 from each were put aside as test data. Since the model does not actually take parameters directly from the data, the SATURN data was just a ground truth to compare against, there was no need for validation data.

During development of the 1DOF simulation, it was only ever run on the training dataset. Only once the simulation was finalised was it run on the test dataset to obtain the final performance metrics.

The RMSE was used as the evaluation criteria as having a small number of situations where the simulation over or under estimates the apogee considerably is undesirable and should be reflected in the performance metric.

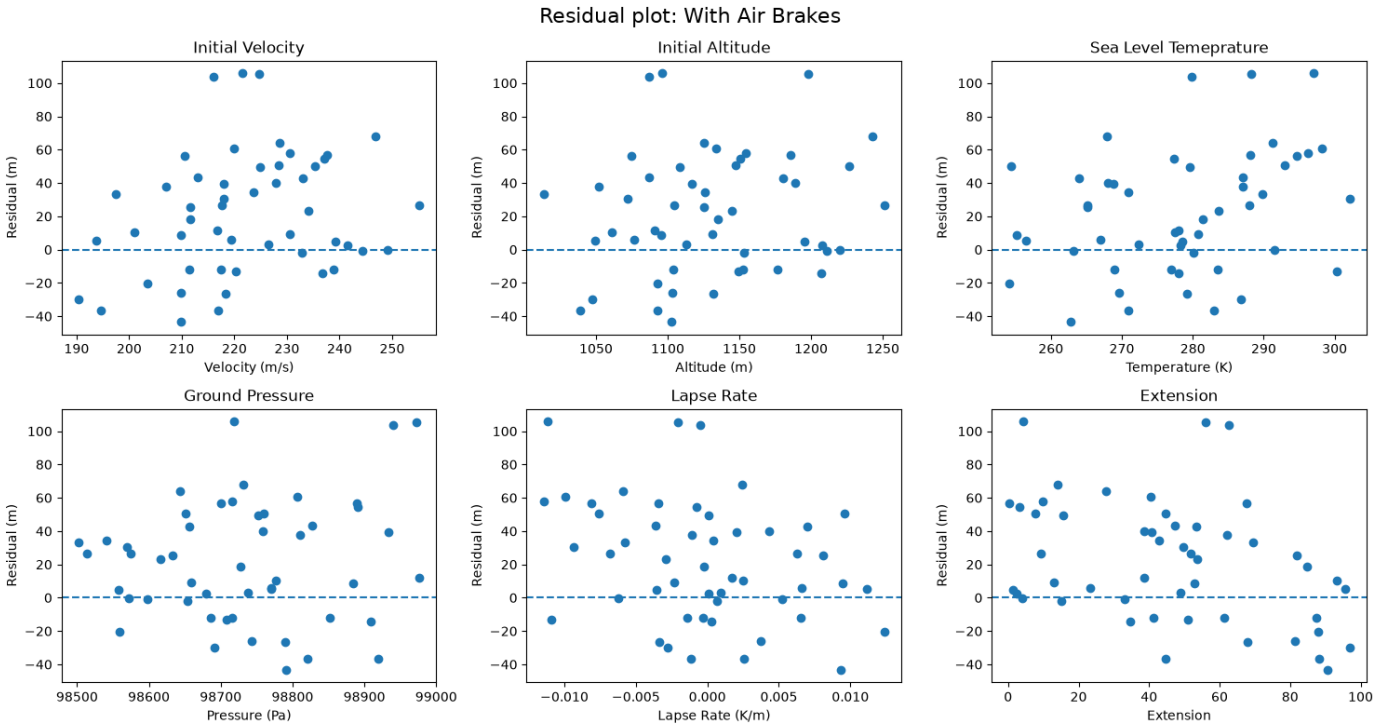


Figure 2: Residual plot of the simulation against the supplied initial conditions in the with Air Brakes dataset

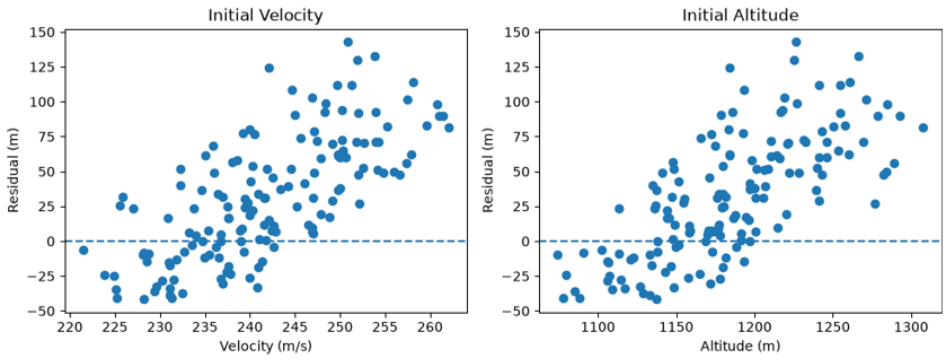


Figure 3: Initial velocity and initial altitude residual plots for the without Air Brakes dataset training data

The 1DOF achieved a final RMSE of 42.14 meters on the test data from the with Air Brakes dataset. As can be seen in figure 2, there are no clear patterns in the residuals and thus each initial condition is being accounted for correctly. However, when run on the without Air Brakes training data linear trends emerge with initial velocity and initial altitude as can be seen in figure 3. Since the altitude and velocity at burnout are highly correlated, these plots likely show the same underlying trend. Upon inspection of specific data points which have high, error an explanation can be formed.

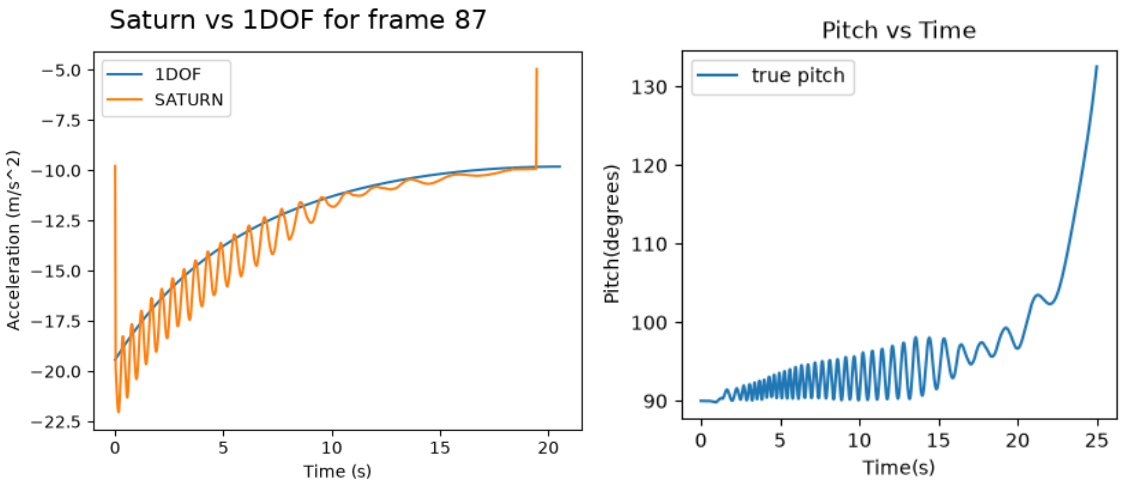


Figure 4: Left: Acceleration vs time plotted for both the 1DOF simulation and original SATURN data from frame 87. Right: rocket pitch across time from SATURN data

As can be seen in figure 4 there are fluctuations in the acceleration in the SATURN data. These fluctuations line up with fluctuations in the pitch of the rocket. Since the 1DOF simulation assumes the pitch of the rocket is always perfectly in line with the air flow, it will always assume the highest possible acceleration at a given point, this can be seen in the graph as the 1DOF acceleration follows the peaks of the SATURN acceleration. Since this inaccuracy in apogee can be attributed to the oscillation, which the 1DOF is not expected to be able to model, the 1DOF is still considered to be physically accurate.

The remaining error can be accounted for by the differences in atmospheric modelling. SATURN uses real life temperature and pressure data with altitude, whereas the 1DOF simulation assumes a linear change in temperature with altitude and a simplified model for pressure. Since pressure and temperature contribute to density and thus drag, there will be small differences in acceleration between the two models.

Control algorithms

On Project Meridian, the control algorithm used a PI method. Onboard the rocket a 1DOF simulation was run and the error between the target apogee and simulated apogee was used as the input to a PI controller. It was not feasible to simulate this algorithm. Since these algorithms are being evaluated on a 1DOF simulation, the error term for this method would be perfect, which would be unrealistic. Furthermore, this method would require running a new instance of the simulation every timestep. Considering the simulation takes 0.04 seconds and has approximately 20,000 timesteps, running this algorithm would take upwards of 15 minutes which was not feasible for this project.

Dataset Generation

Certain control algorithms require training and thus a reference for what extension is optimal given the current conditions. A program was written to generate a random set of initial conditions and find the optimal extensions, which if maintained throughout the whole flight, would reach the target apogee. Randomly selected timesteps from each simulation where then chosen to be saved, along with the optimal extension.

Table 3: Data set parameter ranges

Parameter	**Minimum**	**Maximum**
Initial Velocity	200	260
Initial Altitude	1,100	1,500
Sea Level Temperature	250	310
Sea Level Pressure	98,500	100,000
Lapse Rate	-0.015	0.015

The initial conditions were generated by providing an upper and lower bound for each parameter (Table 3) then choosing value based on a uniform random distribution between those bounds. These ranges were chosen as they were within the bounds of the available SATURN data and produced a roughly equal number of conditions where the rocket would overshoot the target even with maximum extension and where it would undershoot with the minimum extension.

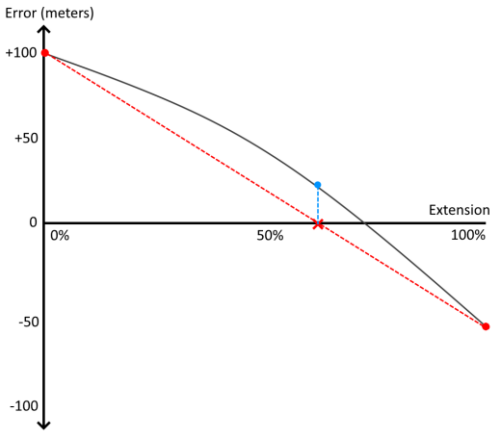


Figure 5: Illustration of the false position method, y axis values are not indicative or true Air Brakes performance

The program used the false position method to narrow in on the optimal extension. A simulation was first run at 100% extension, then at 0% extension. If the 100% extension overshot the target, then the program still saved data from that simulation, except it added a flag to the data to indicate it overshot. Likewise, if the 0% extension undershot the target, then a flag was added to the data to indicate it undershot. This was important for training the models. If both those conditions passed and the target apogee was reachable, then the false position method would be applied. As illustrated in figure 5, based on the error and extension from the previous two simulations, a new extension value is chosen based on where the line between them passes through the axis. This process is repeated until the simulated apogee passes within 0.1 meters of the target.

Training, validation and test data were all generated using the same method. 150 simulations were saved for training, validation and testing. From each simulation, 100 randomly selected timesteps were saved

Inputs

Table 4: Available inputs for control algorithms

Input	**Units**	**Source**
Altitude	Meters	Onboard barometer
Velocity	Meters per second	Onboard Extended Karman Filter
Acceleration	Meters per second squared	Onboard accelerometers
Current extension	Non-dimensional value between 0 and 1	Onboard extension feedback sensor
Ground pressure	Pa	Measured by equipment on the ground
Ground temperature	Kelvin	Measured by equipment on the ground
Lapse rate	Kelvin per meter	Extracted from available weather data before launch

Each control algorithm was presented with the inputs shown in table 4. Not all algorithms utilise all inputs, but they are available regardless.

While the acceleration can be measured in flight, it creates a data leakage problem when using the generated data described above. This is because the acceleration of the rocket depends on the current extension. If the extension, atmospheric conditions and velocity of the rocket are known, the acceleration can be calculated. Likewise, if the acceleration and other conditions are known, the extension that produced that acceleration can be calculated. Since at every datapoint in the training data, the extension is fixed at the optimal value, the acceleration contains information about the optimal extension. Outside the training dataset, the acceleration

only contains information about the current, not optimal, extension which could lead to a model incorrectly using the acceleration. For this reason, acceleration has been omitted as an input to all control algorithms presented.

Outputs

Each control algorithm had a single output, the target extension value which it calculated. However, this output on its own is not meaningful, thus each algorithm was evaluated using the simulation for all the initial conditions created in the testing data. The initial conditions were split into three categories, the ones which could reach the target apogee, the ones which undershot even if the Air Brakes remained retracted and the ones which overshot even if the Air Brakes were extended. For the conditions where the target apogee was reachable, the standard deviation in apogee was recorded. This gave a measure of the consistency of the algorithm. A lower standard deviation means a better performing algorithm. For the undershooting and overshooting conditions, the error between the best possible apogee (Air Brakes fully retracted or extended) and the achieved apogee were recorded. In this case, having values close to zero is most desirable as it means the extension was close to 0% or 100% throughout the flight.

P Controller

The P controller algorithm was created as a baseline to compare other methods against. It took the target apogee and subtracted the current altitude to create an error term which multiplied by a proportional gain to determine the target extension. The proportional gain was tuned via repeated runs of the simulation and the false position method. The quantity being minimised was the average apogee error across the initial conditions in the generated training data. This algorithm had very poor performance, with a standard deviation of 124.76 meters on the test initial conditions. This was to be expected as it had such a limited vision of the current state of the rocket. This was the simplest algorithm tested, only requiring a single subtraction and multiplication to calculate the target extension making it easily implementable on the embedded system in the rocket

Linear Regression

The linear regression algorithm took each of the available parameters, excluding acceleration and current extension for reasons detailed before, and applied a weighted sum. These weights were determined via numpy's least squares regression solver which minimised the error between the calculated extension and the optimal extension found when generating the training data. This algorithm represented an improvement over the P controller algorithm with a standard deviation of 98.22 on the test initial conditions. With the limited number of inputs, this algorithm also required a minimal number of operations in order to execute, making it highly suitable for the real time environment on the rocket.

Revised Linear Regression

The revised linear regression algorithm is functionally the same as the other linear regression algorithm, but with the addition of a velocity squared input. This was done as the drag on the rocket scales with the velocity squared and thus accounting for this nonlinear relationship should improve performance. This algorithm represented a substantial improvement on the original linear regression algorithm, achieving a standard deviation of 39.50 meters. Furthermore, the under and overshoot performance were substantially improved, as can be seen in figure 6 in the performance and comparison section. Like the original linear regression algorithm, this method requires a minimal number of operations to calculate and is thus well suited for the real time environment on the rocket.

Support Vector Regression

The support vector regression algorithm was implemented using an RBF kernel and was trained on the saved timesteps from the training data. C, epsilon and gamma parameters were tuned to find a balance between final RMSE and number of support vectors. Final values were C=2, epsilon=0.1 and gamma=0.1667. Despite being more flexible, this algorithm achieved only a marginal improvement of the linear regression methods a standard deviation of 32.96 on the test initial conditions. This algorithm is not suitable for use in the time and storage limited environment of the rocket, as it required on the order of 1500 support vectors.

Neural Network

A neural network was implemented using pytorch and its inputs are normalised versions of the available inputs. It has a single output node, which is the target extension. The network was trained using the time step training data, meaning back propagation could be used to tune the network weights quickly. Each network configuration was trained for 1000 epochs. Another training method using a natural selection approach was consider but due to the number of simulations required for each generation this process would have taken too long given the timeframe for this project. The final network consisted of a single hidden layer of 16 neurons. This achieved a standard deviation of 14.78 meters. More complex networks with more hidden layers were tested but these models heavily overfit to the training data and as such performed worse on the validation data than simpler models. While this network is simple and could be executed easily on the rocket, it will only ever be as accurate as the training data. The 1DOF simulation is highly idealised and thus generating training data was fast but for a more complicated simulation this would be an expensive process.

Performance and Comparison

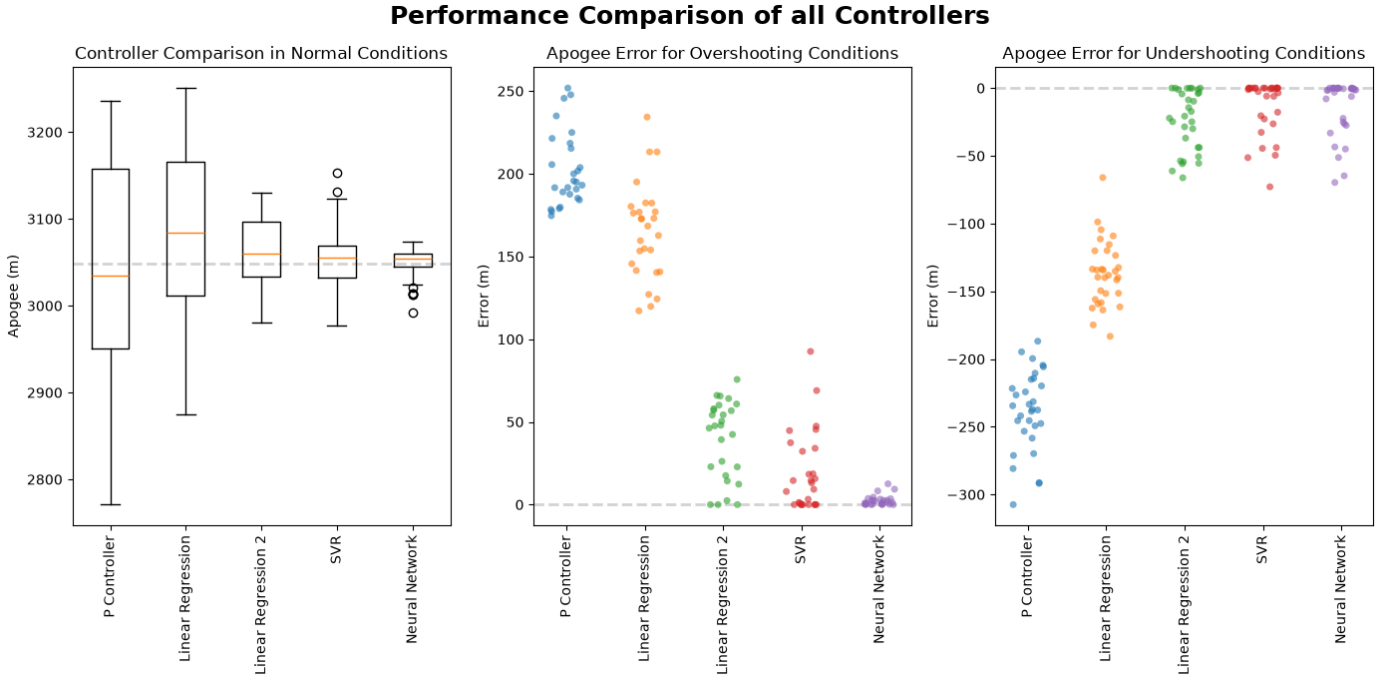


Figure 6: Performance comparison of all algorithms

As can be seen in figure 6, the neural network was the most performant. Achieving apogees well centred around the target mean with very low variation. The P controller and linear regression models on the other had extremely poor performance. Under initial conditions where the rocket couldn't reach the target apogee even with full extension or retraction, these algorithms consistently either extend too far or not enough. This is evident by the fact in the apogee error for over and under shooting conditions no points approach the ideal error of zero.

Overall, either the revised linear regression model or the neural network model would be the best to further. One of the issues however of the neural network model is traceability, if the algorithm behaves strangely in simulation or actual flight, figuring out where that behaviour is coming from is near impossible. This is a highly undesirable characteristic especially for a rocket which can only launch a couple times a year.

Future development

Testing these algorithms with an extension delay included would provide a more accurate picture of how these algorithms would perform in real life. The functionality for this is already present in the code and would just need to be enabled. Implementing artificial sensor noise and observing how each algorithm responds could be done by having an intermediate function which adds noise to the inputs before passing them on to whichever algorithm

is to be checked. Switching the simulation to a 2DOF which models the altitude and pitch of the rocket may allow the oscillating behaviour to be accounted for and thus improve the accuracy of the simulation.

Using the developed simulation as a control algorithm itself could be viable. In generating the training data, it was found using the false position method the simulation converged on an optimal extension within at most four iterations. The simulation would need to be timed on the real hardware to ascertain the feasibility of this idea. The neural network model could also be developed further. For more complex networks it was found the algorithm overfit to the training data heavily, increasing the amount of available training data could mitigate this and allow a for a more performant algorithm. A physics informed neural network or something similar could also improve performance without the need for extra training data.

Conclusion

The 1DOF simulation captured a substantial portion of the underlying physics, however there was still behaviour in the ground truth data from SATURN it couldn't account for. Namely oscillation which lead to an increase in drag. Upon testing multiple control algorithms on this simulation, it was found a linear regression approach with included non-linear inputs or a neural network model both provided good performance. Achieving a low standard deviation and average apogees centred around the target. Either model is simple enough to run on the rocket, however the neural network algorithm would have problems with traceability if problems ever needed to be diagnosed. Areas to develop further include improving the accuracy of the simulation, implementing an extension delay and simulated noise, and giving more training data to the neural network.

AI Use

AI, specifically Claude Sonnet 5.5, was used to generate parts of the code for this project. Whenever AI has been used, the relevant code is bookended by comments containing lots of semicolons as well as a comment about what AI generated the code and whether it was modified manually after the fact. The simulation was written almost entirely by hand, except for type and value checks for some functions. AI was used more heavily in developing the control algorithms, where it would generate entire controller and training functions, and code to plot results. The logic behind each controller, with the exception of the neural network controller, was determined manually. To ensure the code generated was correct, all outputs of the AI were read over and the outputs of the code generated were checked for consistency with the desired logic. AI was used for the control algorithm functions as they have very clearly defined inputs, outputs and underlying logic, meaning getting a correct output from the AI was simple. AI was also used for the plotting functions for the same reasons, as what really mattered from those plots was getting a better understanding of the system.

When used in this limited capacity, AI did improve the speed at which this project could be developed. Rather than spending upwards of 10 minutes programming a plotting function, the plot could be generated within seconds and the information contained in that plot could be acted on sooner. I would not be comfortable giving the AI more control and access to the code than this in fear of losing understanding of how the code works.